目录

01 检索前置优化：查询改写

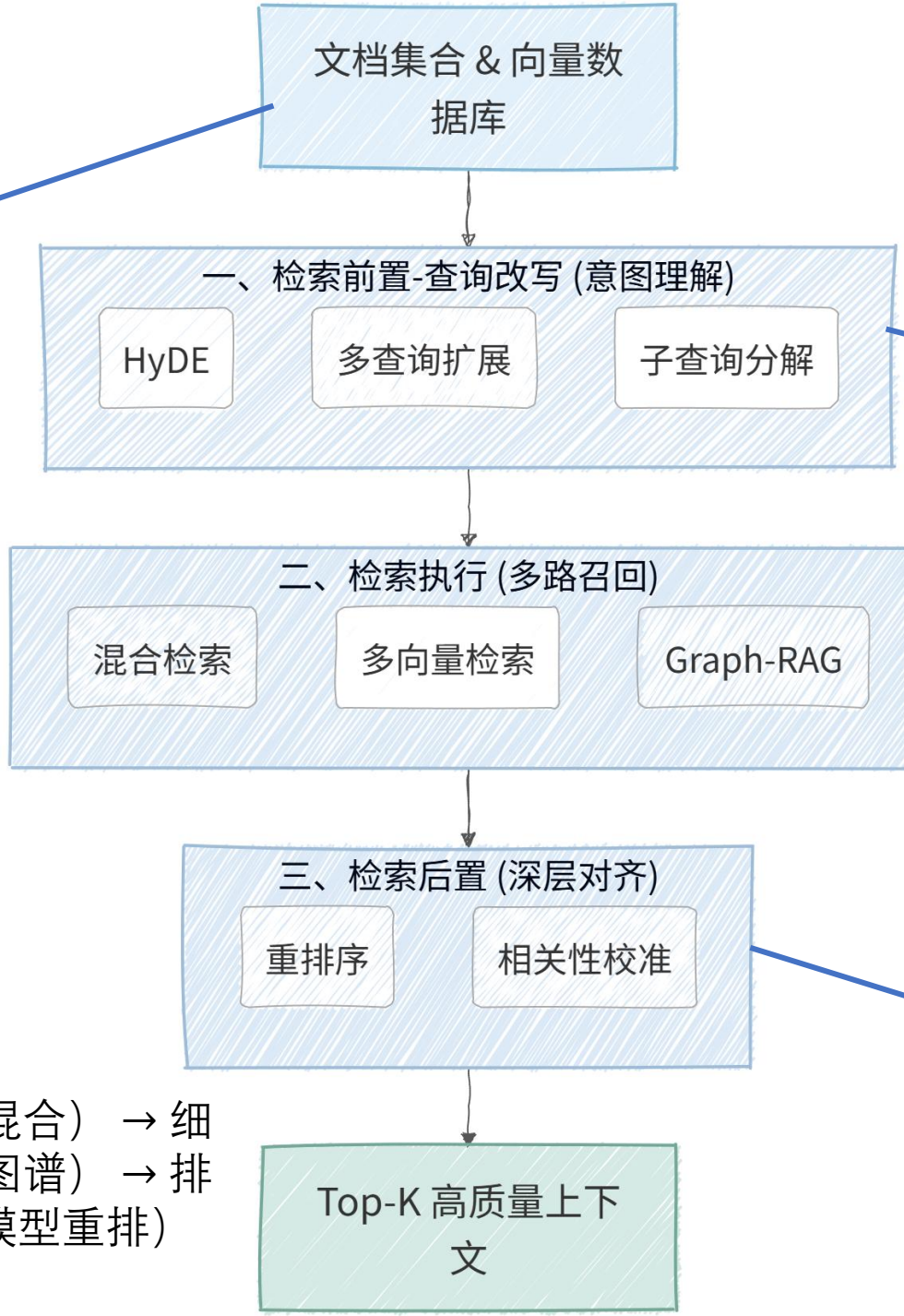
02 检索执行：混合检索

03 检索执行：多向量与图检索

04 检索后置：重排序与校准

框架图

- 语义鸿沟
- 检索死角
- 粗糙排序



转化增强查询，跨越鸿沟

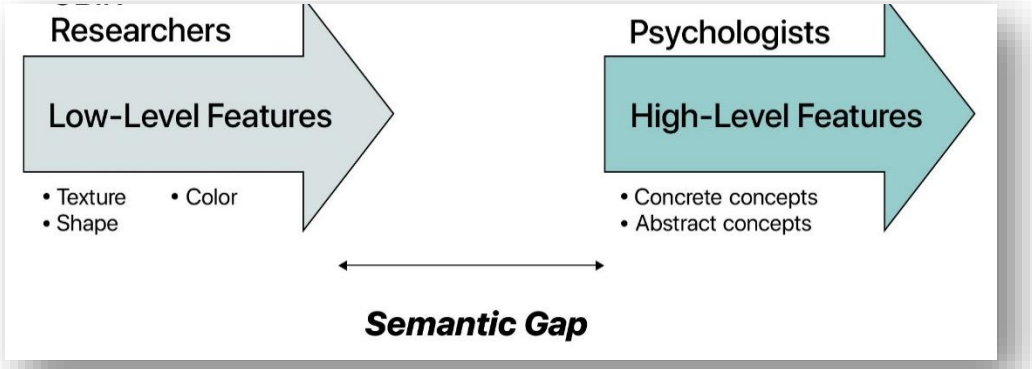
加强细致搜索

精准排序

提问太笨（1.改写） → 搜不准名词（2.混合） → 细节丢失（3.多向量） → 缺乏大局观（4.图谱） → 排序粗糙（5.重排） → 结果推理（6.大模型重排）

核心问题

场景还原：语义错位导致检索失败



- 用户提问：“如何解决大模型在语言生成时出现的‘幻觉’问题？”
- 知识库内容：包含“通过RAG技术提升金融问答系统事实准确性”的文档
- 检索结果：由于“幻觉”与“事实准确性”字面匹配度低，关键文档被遗漏

- **语义鸿沟**

自然语言和书面用语存在差别，导致在向量空间无法对齐

- **信息丢失**

传统架构将长文档压缩为唯一低维向量，丢失信息细节

- **缺乏全局视角**

检索仅聚焦于局部，忽视信息中的相互关联与全局宏观认识

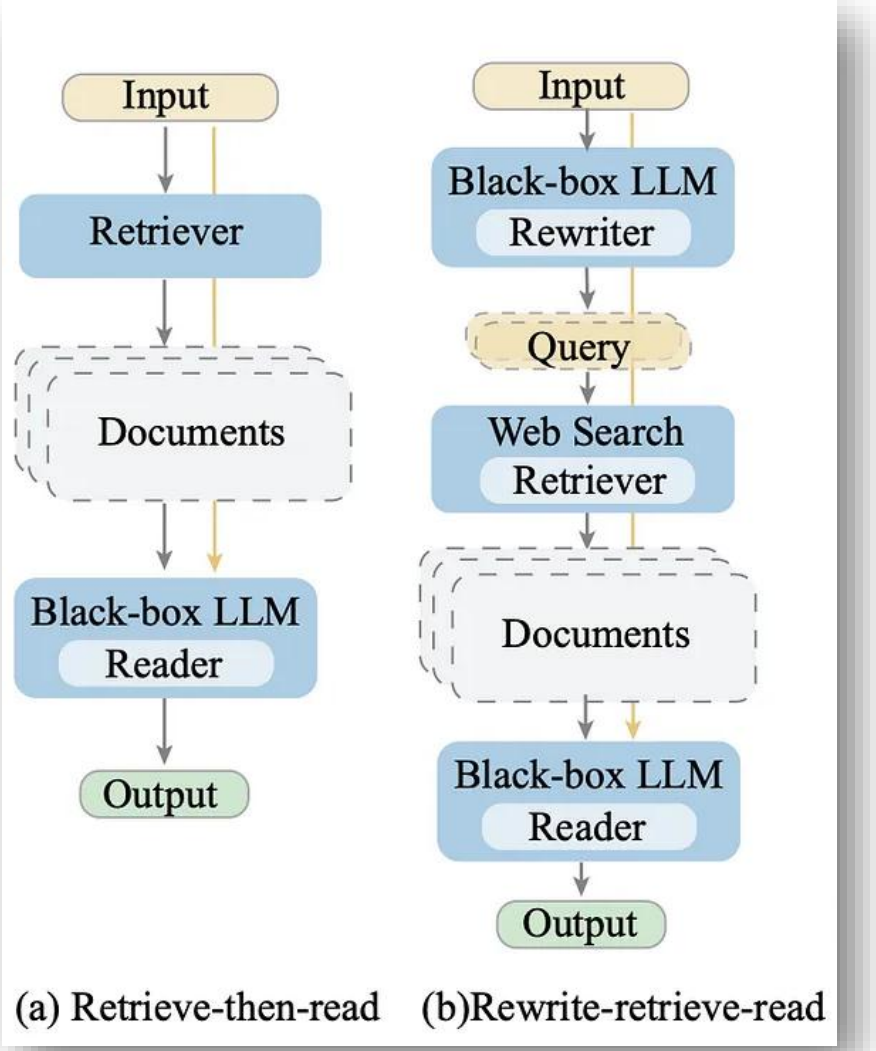
PART 01

检索前置优化：查询改写

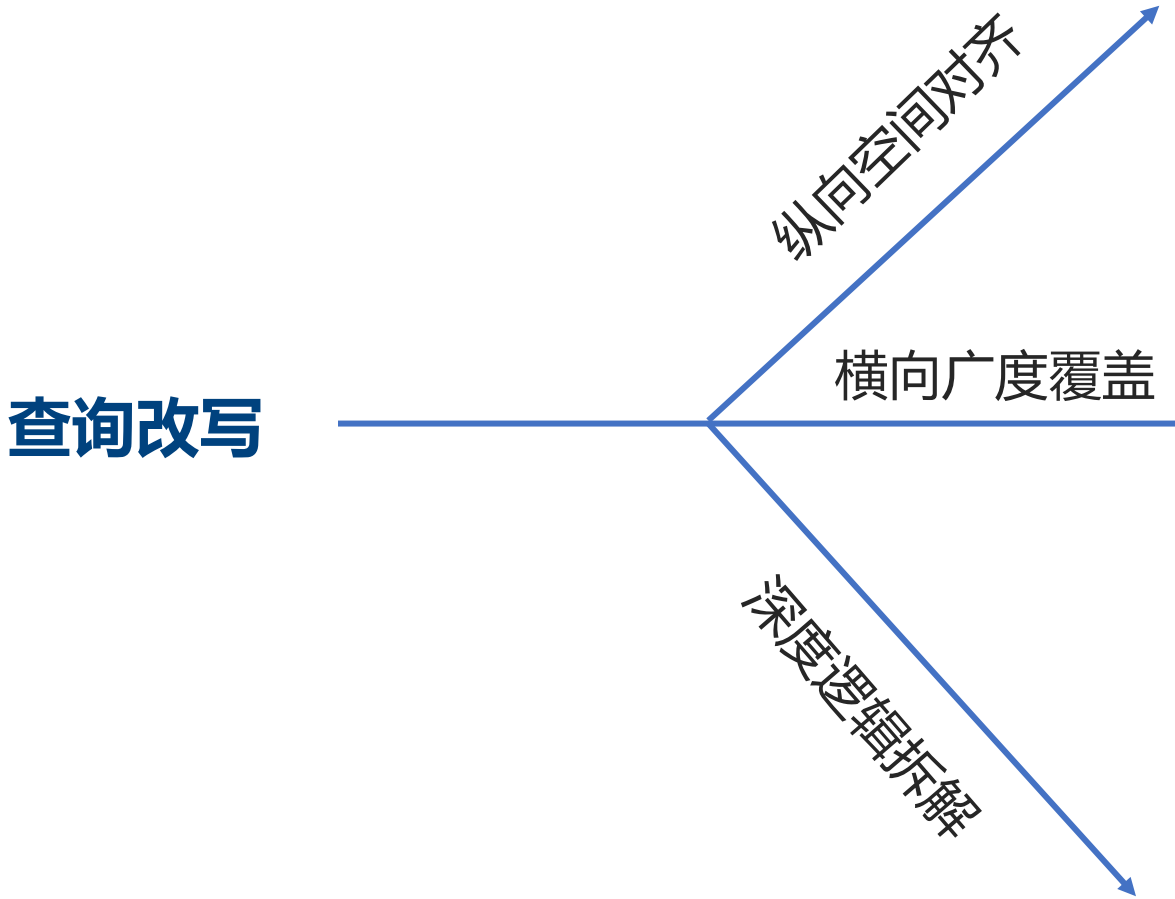
查询改写与语义对齐

查询改写

- 用户的提问**口语化或简短**，而知识库中文档使用的**学术词汇或术语**
- 引入**大语言模型 LLM** 作为查询前的翻译官，对用户的原始查询进行纠错、扩写或同义改写
- 本质：query → query



查询改写与语义对齐



HyDE假设性文档嵌入

针对问题生成一份假答案，用假答案匹配真文档，跨越问题与文档的空间鸿沟

多查询扩展

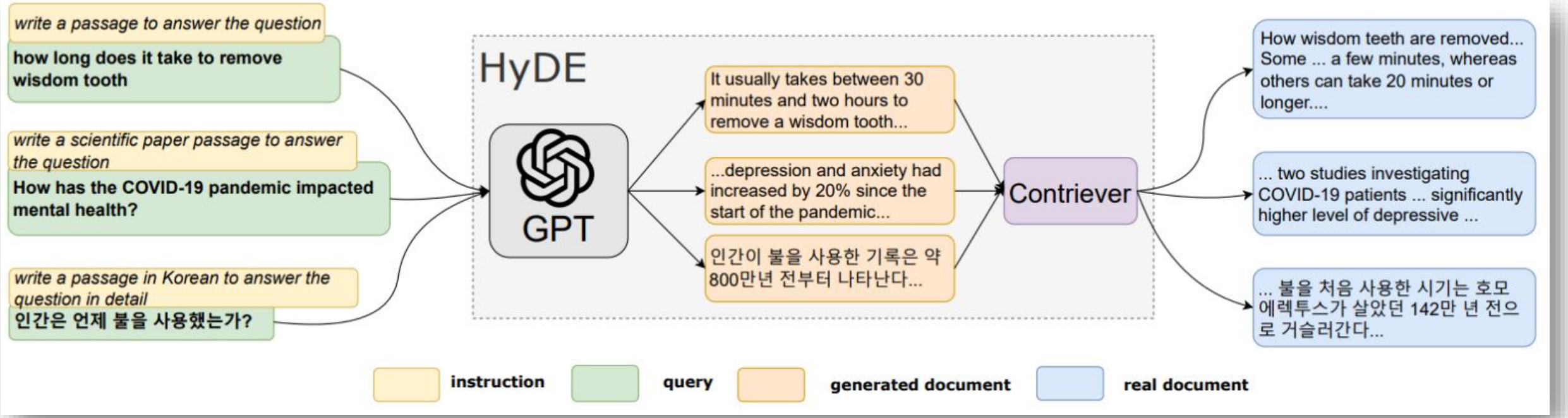
针对多视角问题，一分为多并发检索，用 RRF 融合，求完整

子查询分解

针对多步复杂意图，长句拆短句、大任务拆小任务，顺序执行，求深度

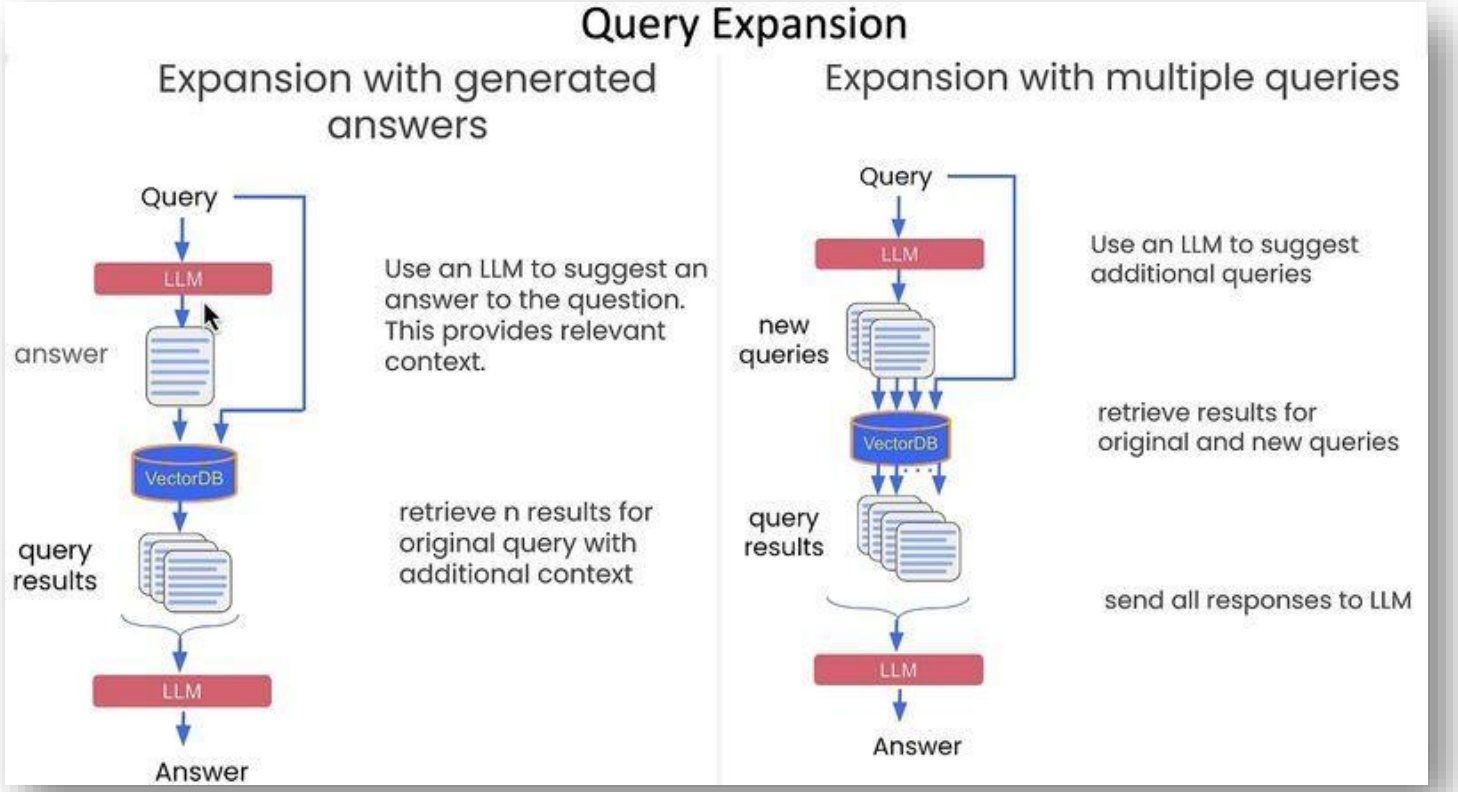
HyDE假设性文档嵌入

- 传统向量检索痛点：直接计算 **问题向量** 与 **文档块向量** 的余弦相似度，而问题常为 **短句/疑问句**，在向量空间中的特征分布与 **长句/陈述** 不属于一个高维类别。即便内容相同，因为句式和表达方式不对齐，分数不高
- 核心思想：用 **答案** 找 **答案。**根据原问题，用LLM先生成一个假设性答案，此时，这个假答案的句式、术语和结构已经完全属于答案空间了。



多查询扩展

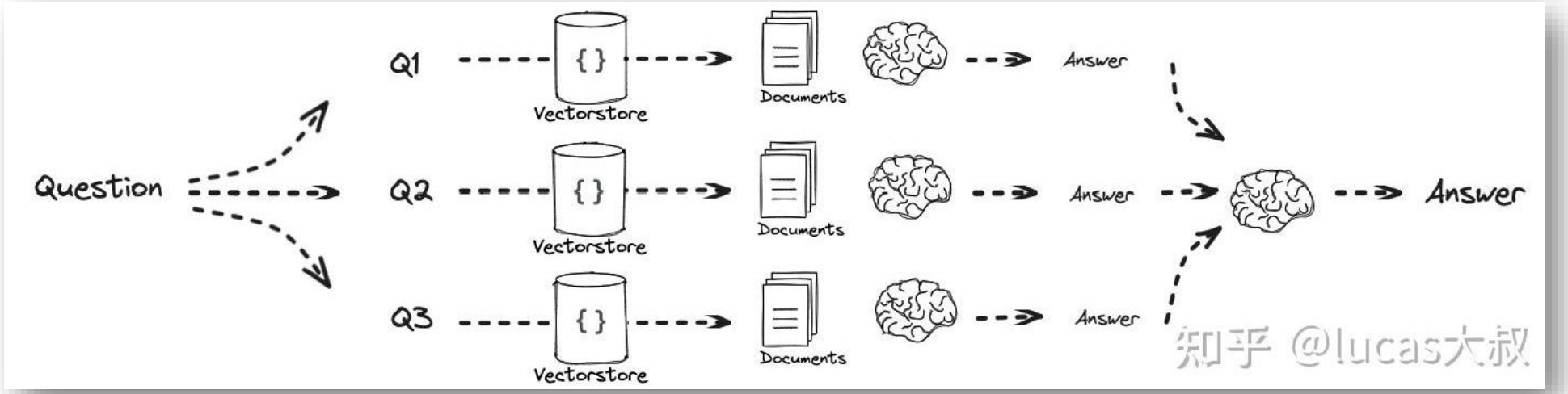
- 单一查询痛点：原始提问和改写后提问，通常只针对某个问题或某个切入视角，很难完整匹配到多维的相关的答案，导致信息缺漏。
- 核心思想：将 **一个问题** 扩展为 **多个问题。**利用LLM的语言发散能力，针对用户的同一个核心意图，从同义词、上下游、多维视角等方向衍生为 3~5 个并列的新查询。



- 原问题： "大模型为什么会产生幻觉"
- 扩展1："大语言模型产生幻觉的原因"
- 扩展2：“LLM事实性错误的来源"
- 扩展3： "生成式AI出现虚构内容的机制"

子查询分解

- 复杂查询痛点：用户的原始提问往往不是一个单一的知识点，而是包含多步推理、跨时间对比、或者条件组合的复合型查询，向量会被多个关键词污染，得到包含噪音的结果。
- 核心思想：将 **一个复杂问题 按顺序、因果关系** 分解为 **多个子问题。**利用LLM的逻辑解构能力，扮演一个架构师，将一个复杂的母问题，按照时间轴、因果关系，拆解为一组子问题链。



- 原问题： "大模型为什么会产生幻觉"
- 分解1："大模型是如何生成下一个 token 的"
- 分解2："训练数据为什么会导致事实错误"
- 分解3： "为什么缺乏外部知识会加重大模型幻觉"

PART 02

检索执行：混合检索

检索执行策略

检索执行

混合检索

稀疏检索(BM25)

依托词频、信息熵及惩罚机制进行精确关键词匹配，专注特定专有名词与数字细节

稠密检索

将文本映射至低维高密空间，作为跨越词合围的语义感知者，利用余弦相似度捕捉用户模糊的宏观意图

细粒度检索

多向量检索

不使用长文档粗暴压缩，文本内的每一个 Token 都拥有独立表征，利用轻量级延迟交互进行词对词细粒度检索

图增强检索

利用大模型将非结构化文本，重组为包含实体与关系的全局知识网络，通过拓扑关系分析关联性

稀疏检索(BM25)

- 语义检索痛点：强行将文本压缩至低维高密空间，会把低频出现的生僻专有名词当作噪音抹平，导致无法检索到具体细节。

- 核心思想：基于词项是否出现，而不是语义，属于词法匹配，其中BM25是TF-IDF的升级版。

- TF (Term Frequency) — 词频

$$TF(t,d) = \frac{\text{词 t 在文档 d 中出现次数}}{\text{文档 d 的总词数}}$$

- IDF — 逆文档频率（重要性）

$$IDF(t) = \log \frac{N}{1 + \text{包含词 t 的文档数}}$$

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

总结：一个词，在当前文档出现很多次，
但在其他文档很少见，它就非常重要

稀疏检索(BM25)

$$BM25(D,Q) = \sum_{i=1}^{n} IDF(q_i) \cdot \frac{f(q_i,D) \cdot (k_1+1)}{f(q_i,D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})}$$

- TF-IDF：理想化模型，认为TF词频是线性增长，实际上词频收益逐渐减弱。其次长文档词更多，更容易命中导致偏向长文档。
- BM25：引入词频饱和，即先增长很快，后面逐渐放缓，边际收益递减。并且通过长度归一化，惩罚长文档的优势。

- IDF — 逆文档频率，惩罚常见词
- k1 — 词频饱和度控制
- b — 文档长度归一化

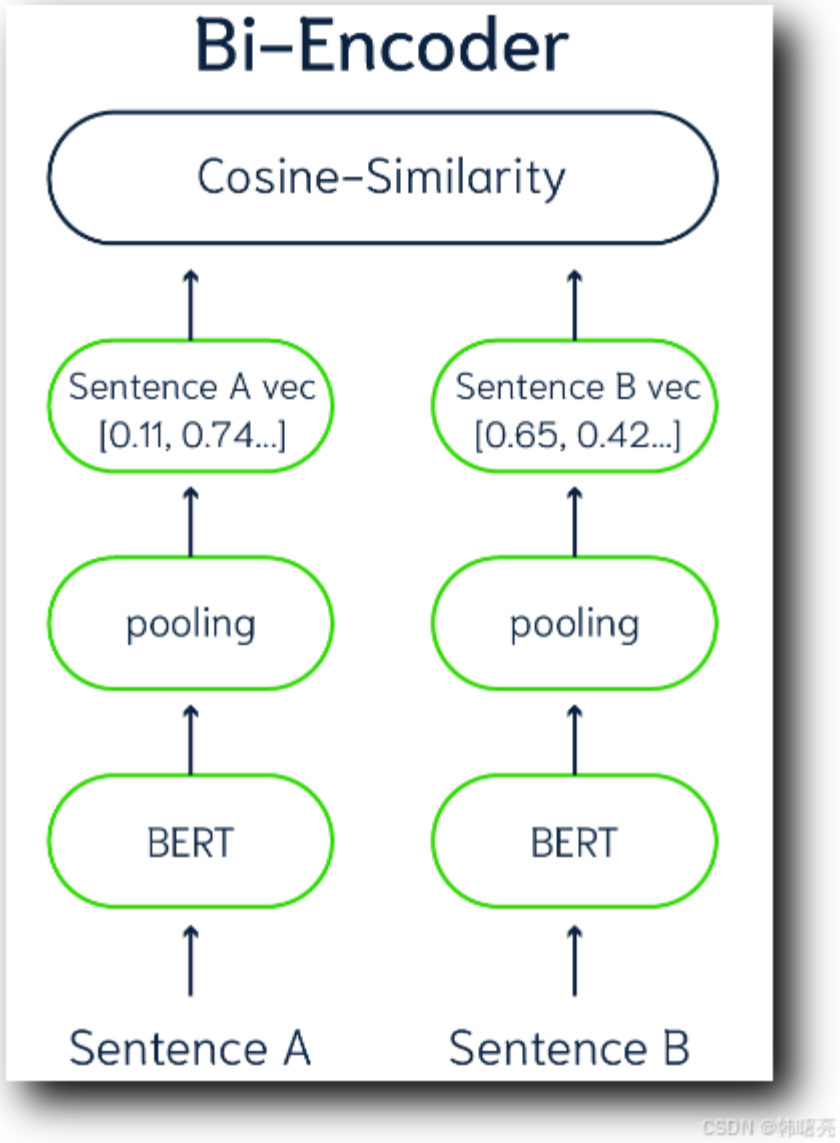
可以表示为稀疏向量，但是不属于向量检索，

关注词是否出现，出现多少，词是否稀有

无法理解匹配语义

稠密检索

- 核心思想：基于语义向量的检索。利用深度学习双塔架构，将文本映射至固定的低维高密空间，利用余弦相似度捕捉跨词汇的宏观大意
- 稠密：向量大部分维度都非0
- 把文本映射到语义空间里做最近邻搜索

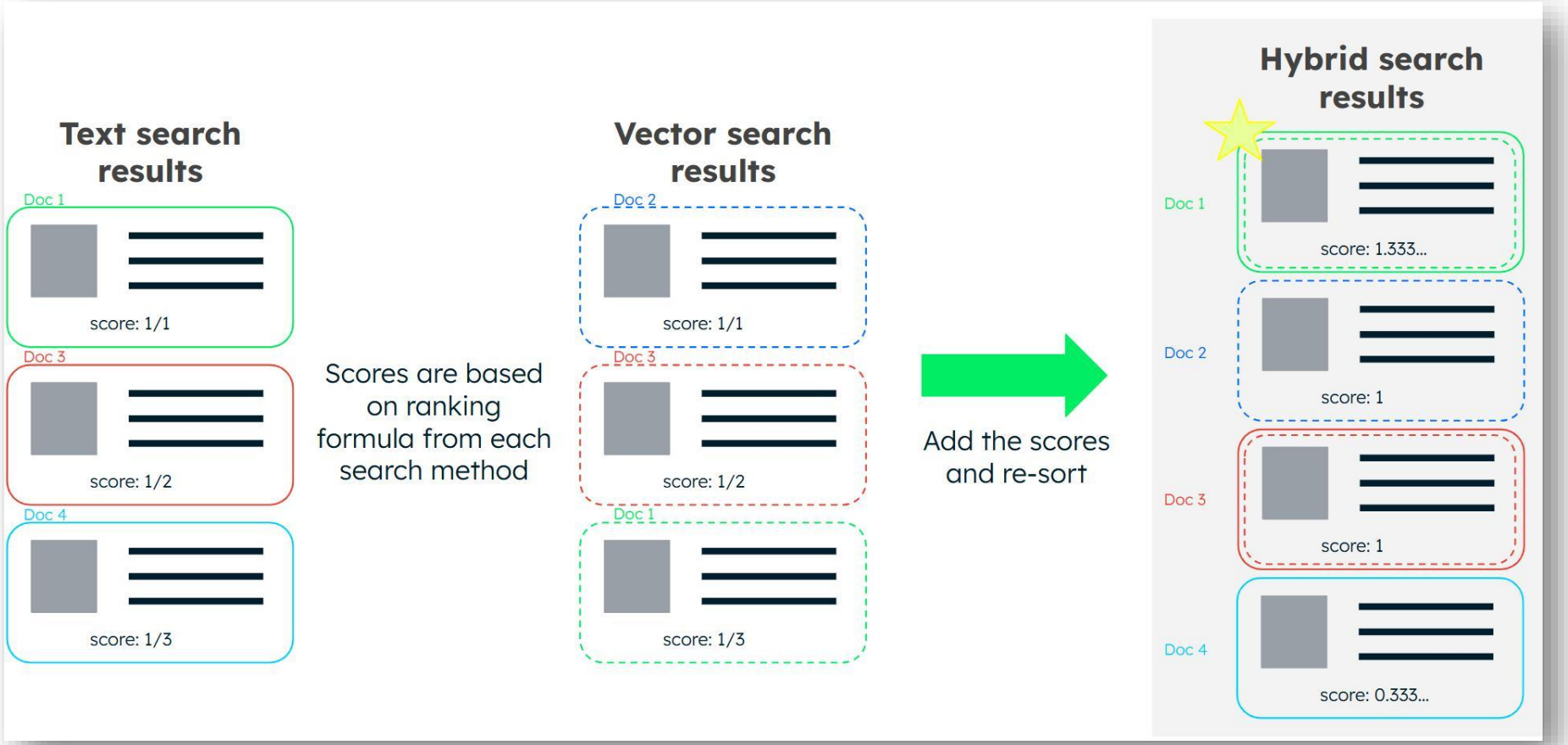


CSDN @韩曙亮

混合检索

$$RRF_Score(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)} \quad (默认\ k = 60)$$

- 混合检索：稀疏检索 + 稠密检索，BM25（关键词匹配）+ Dense Retrieval（语义匹配），两者进行RRF融合



- RRF：把多个检索系统的排名做加权倒数求和，用来稳定融合 BM25 和 Dense 的结果
- 一个文档在多个检索结果里排得越靠前，分数越高

PART 03

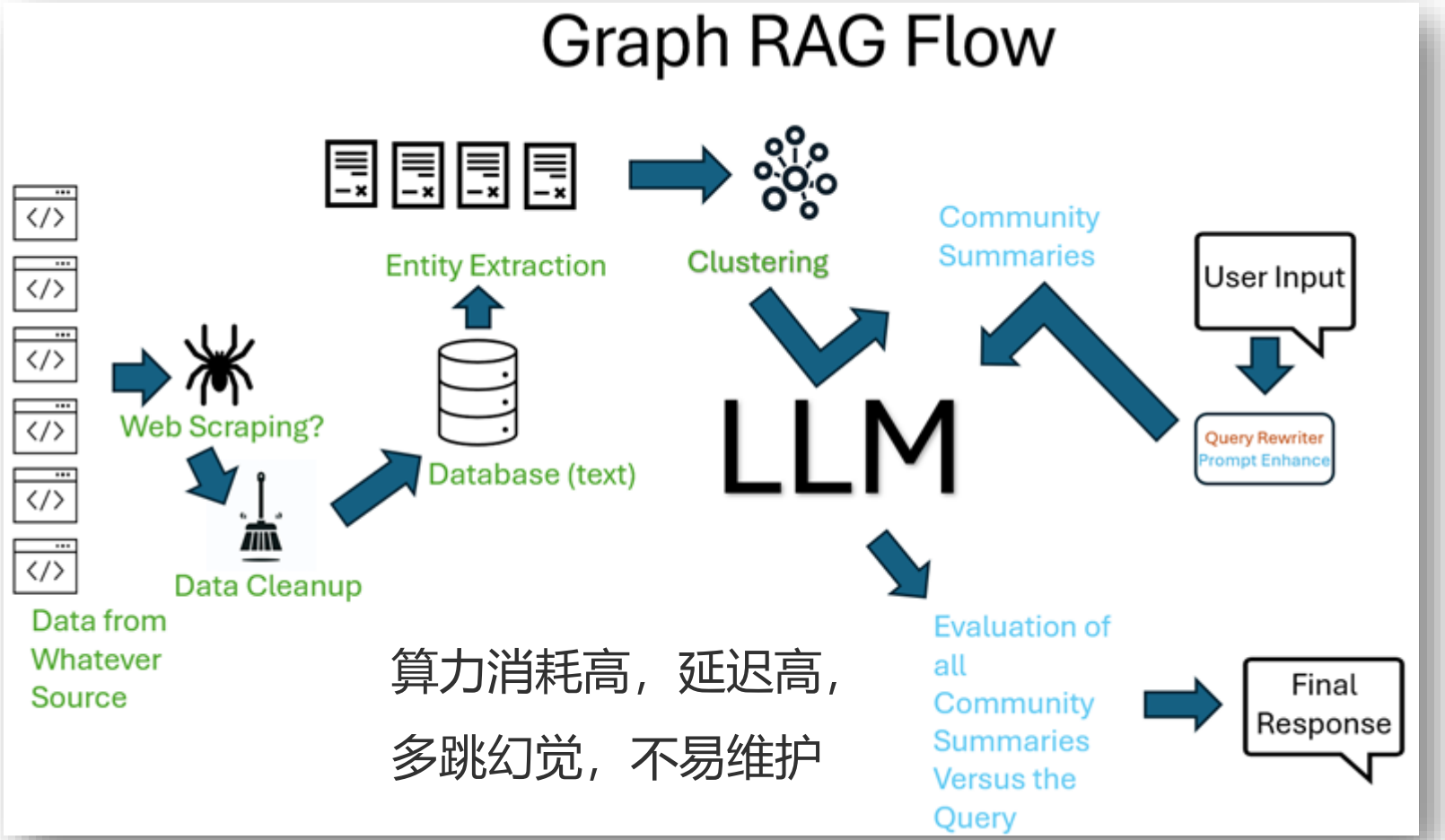
检索执行：多向量与图检索

图增强检索Graph RAG

- 传统RAG核心痛点：检索算法在底层处理的数据结构全部都是**孤立的文本块**，文本之间的关系被切断了，无法回答总结性问题，无法跨文档推理。
- 核心思想：不仅检索文本，还要检索**关系**。在非结构化文本上，把知识组织成包含实体与关系的图结构。

核心步骤：

1. **结构化建图**：抽取实体与拓扑关系，构建知识图谱，社区发现算法，大模型为社区生成概括
2. **图检索**：顺着拓扑关系（边），横跨不同文档，组成因果链
3. **LLM生成**：检索内容由大模型概括总结



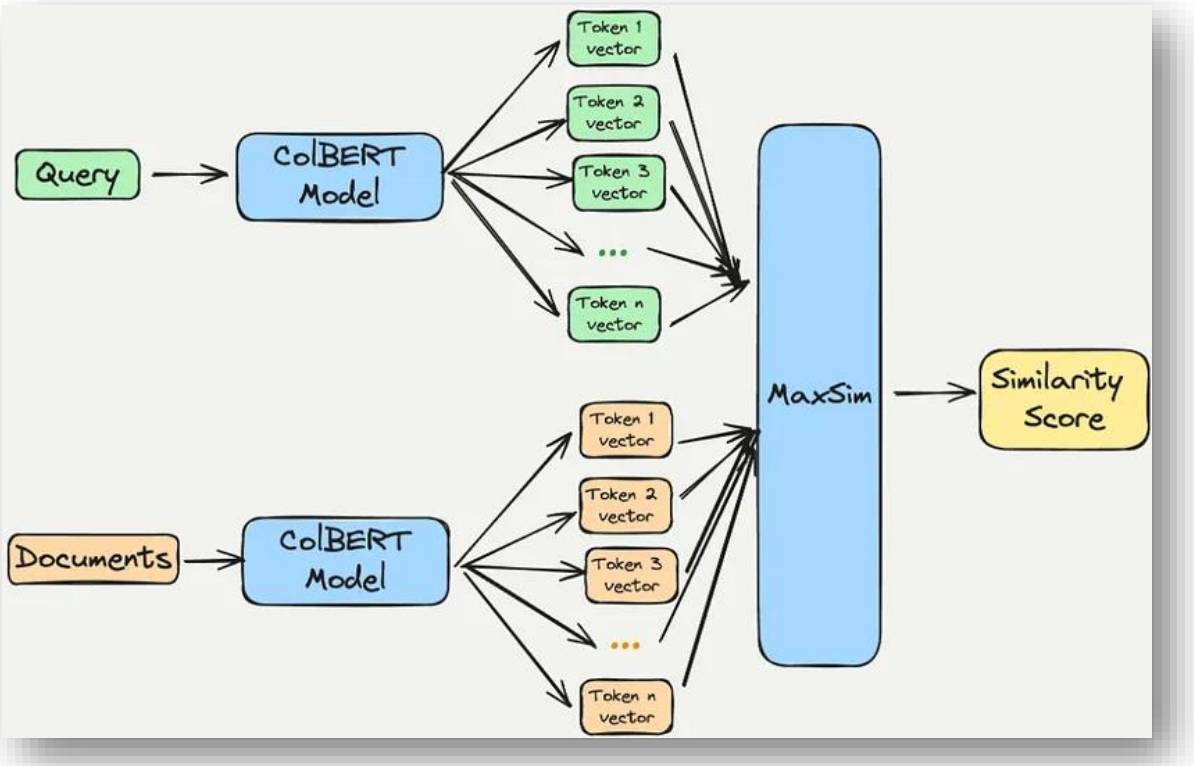
多向量检索（Col BERT）

- 单向量检索：信息压缩瓶颈，这种高度概括的压缩，会导致长文章中间隐藏的某一句核心信息被抹平

- 核心思想：一段文本用多个向量表示，而不是一个向量，让提问与文档内的每一个 Token 独立表征

$$Score(Q, D) = \sum_{i \in Q} \max_{j \in D} \left(E_{q,i} \cdot E_{d,j}^{T} \right)$$

- BiEncoder：粗粒度
- CrossEncoder：太慢



- 矩阵化表征：提问里的每一个Token、长文档里的每一个Token，都拥有自己独立的、上下文相关的特征向量，提问Q变成了一个Eq矩阵，文档D变成了Ed矩阵

- 延迟交互机制：在检索的最后阶段让两个矩阵进行轻量级的MaxSim最大相似度算分

PART 04

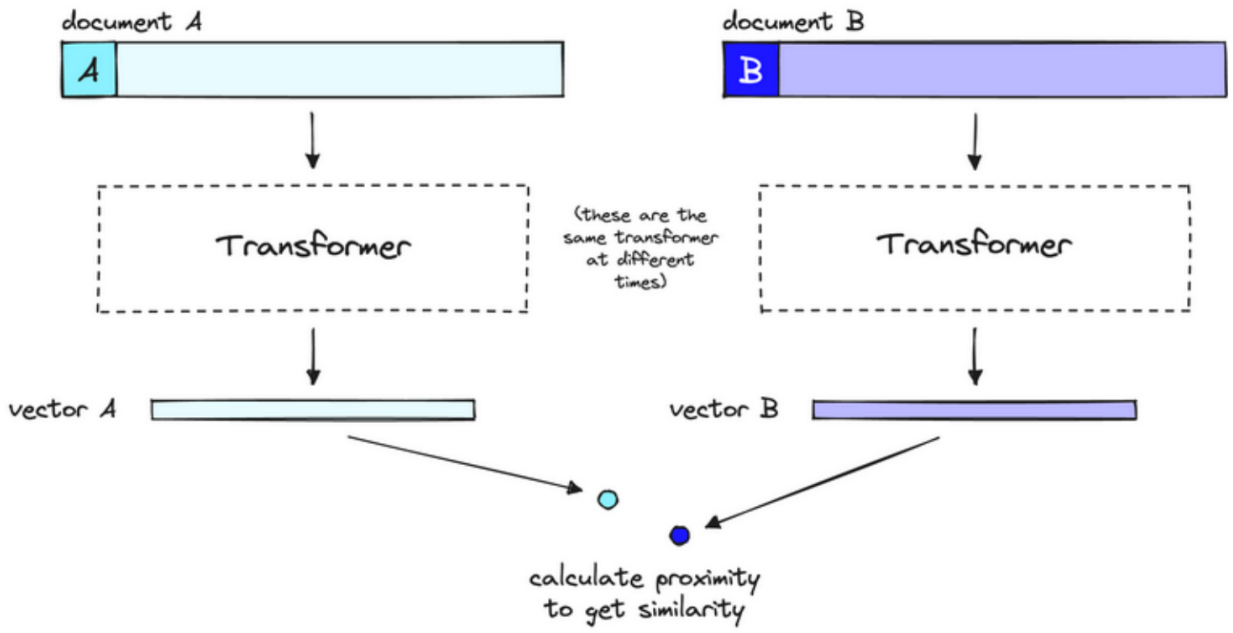
检索后置：重排序与校准

粗排的局限与“中间迷失”现象
——我们为什么要引入重排序

向量粗排的表征瓶颈一

传统的向量检索（双塔模型/Bi-Encoder）将用户问题与知识库文档分别独立压缩为稠密向量，在计算相似度前，问题与文档缺乏底层“词与词”的交流。
这种机制使得粗排过程仅仅是宏观语义分布的匹配，极易遗漏局部的特定名词、数字和细节逻辑。虽然召回了大量文档，但往往“形似而神不似”，无法直接作为精准答案的上下文。

向量相似度 ≠ 语义相关性



A bi-encoder model compresses the document or query meaning into a single vector. Note that the bi-encoder processes our query in the same way as it does documents, but at user query time.

粗排的局限与“中间迷失”现象——我们为什么要引入重排序

向量粗排的表征瓶颈二

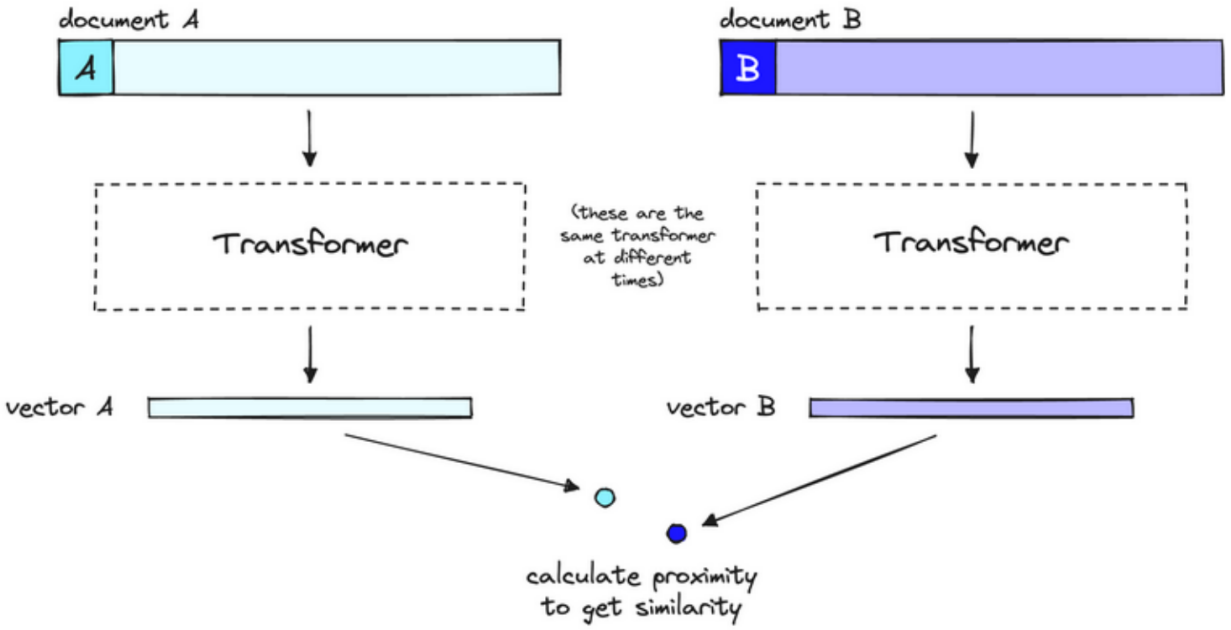
如果较低位置的文档包含了有助于大语言模型更好地形成回答的相关信息，该怎么办？

一个简单的方法是增加返回的文档数量（即增加top_k值），并将它们全部传递给大语言模型。但是这样做有一定的条件：

第一，这样做的一个劣势，就是需要消耗更多的token，意味着成本的增加。

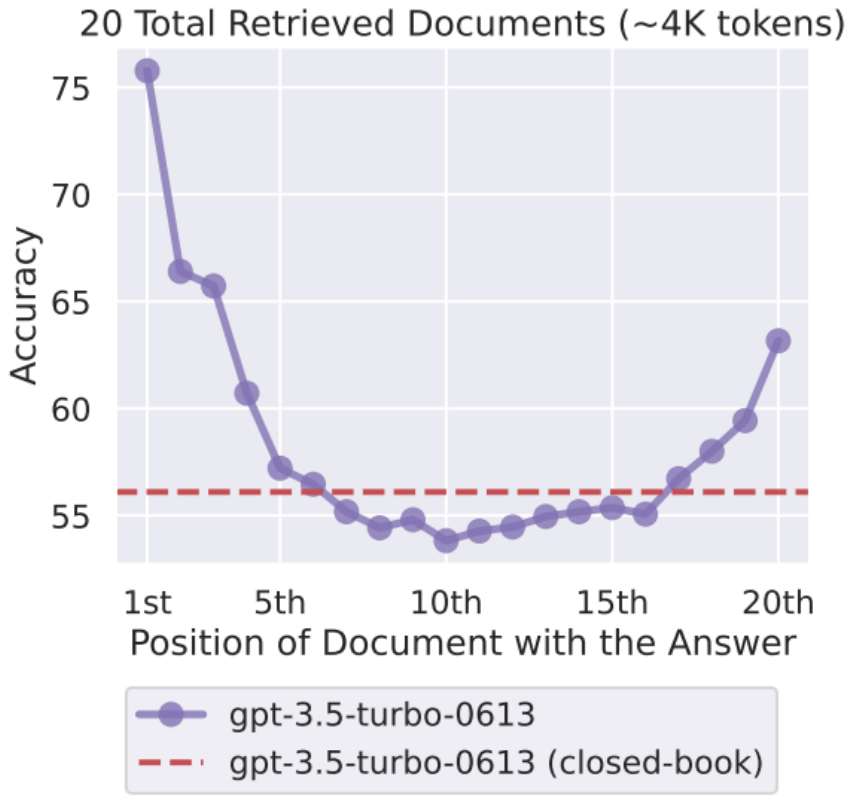
第二，尽管大模型拥有高达100K Token的巨大上下文窗口，理论上可以包含大量文档，但我们仍然不能返回所有文档并填满上下文窗口来提高召回率。

第三，当我们在上下文窗口中填充过多内容时，会降低大语言模型在该窗口中检索信息的能力。研究表明，当上下文窗口被过多Token填满时，大语言模型的回忆能力会受到影响。此外，过度填充上下文窗口还会使模型较难按指令执行，因此，这种做法是**不可取**的。



A bi-encoder model compresses the document or query meaning into a single vector. Note that the bi-encoder processes our query in the same way as it does documents, but at user query time.

粗排的局限与“中间迷失”现象
——我们为什么要引入重排序

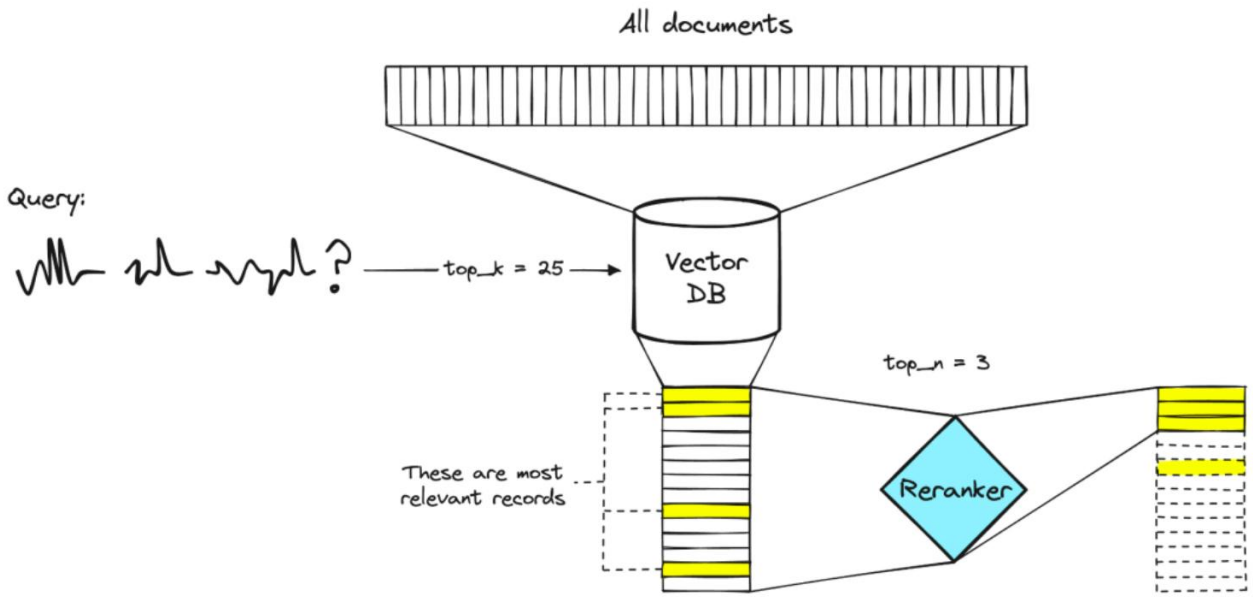


大模型的 "Lost in the Middle"

斯坦福大学的学术研究证明，当赋予大语言模型（LLM）极长的上下文时，它会表现出明显的注意力偏移：高度关注开头和结尾的信息，而忽略中间的内容。

这意味着，如果我们把粗排捞出的 Top 100 篇文档直接丢给大模型，真正含有答案的文档如果排在第 50 位，将被彻底无视。因此，我们必须引入重排序漏斗，将最核心的文档强行置顶。

先初步了解什么是重排序算法



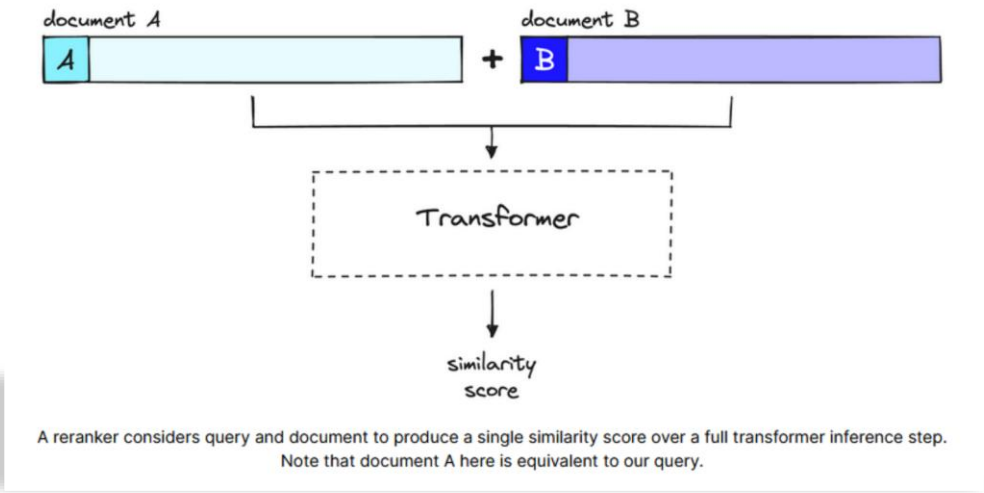
重排序算法是现代搜索增强系统（RAG）中的**“第二阶段精筛漏斗”**。它接收第一阶段（如支持粗排）初步的候选文档，由此与用户提问的“真实相关度”进行精准的重新打分，把最匹配的文档强行置顶。

再看核心机制：重点介绍交叉编码器 (Cross-Encoder)

重排序算法引入了 **交叉编码器 (Cross-Encoder)**，实现了从模糊召回到精准判别的范式跃迁。

交叉编码器抛弃了独立生成向量的做法。它将用户的提问 (Query) 和候选文档 (Document) 在物理层面上直接拼接：[CLS] Query [SEP] Document，并作为一个完整的长序列输入到深层神经网络中。

此时，提问中的每一个字都会与文档中的每一个字发生密集的 **自注意力 (Self-Attention) 碰撞**，实现极其细粒度的语义对齐，最终由 [CLS] token 输出精准的相关性得分。



A reranker considers query and document to produce a single similarity score over a full transformer inference step.
Note that document A here is equivalent to our query.

交叉编码器 (Cross-Encoder)　vs　双编码器（Bi-Encoders）

对比维度	支撑粗排：双塔模型（Bi-Encoder）	重排序：交叉编码器（Cross-Encoder）
核心机制	**“背对背”独立编码** 问题与文档分别独立生成处理，只需在最后进行简单的余弦相似度计算。	**底层深度交互** 问题与文档物理拼接，通过Transformer的自注意力机制（进行逐字碰撞。
计算复杂度	**极低（线性）** 文档支持可离线预计算。在线对接时计算一次支持点积，复杂度为O(1)。	**极高（指数级）** 无法预计算。在线推理时间复杂度与文本总长度的平方成正比O((N+M)^2)。
匹配精度	较低 信息被强行压缩进单一管理，很容易丢失具体的数字、术语等细粒度信息。	较高 不仅理解宏信号，还能精准捕捉上下文语境、同义替换和逻辑因果关系。
响应速度	**极快（数十级）** 能够瞬间完成对百万/千万级文档库的搜索。	**较慢（数百毫秒至秒级）** 由于实时计算量庞大，通常会存在明显的延迟负载。
在RAG中的定位	**第一阶段：海选求全（粗排搜索）** 建立“粗筛漏斗”，从海量知识库中快速召回前100名的粗筛候选集。	**第二阶段：精排求准（精准排名）** 实行“精选漏斗”，对前100名重新严格要求打分，摘取真正的前3名喂给大模型。

主流重排序模型介绍

一：轻量级神经网络精排

技术特点：利用BERT等Transformer预训练模型，将问题与文档拼接后进行物理级特征交叉（自注意力机制），输出直接的相关性打分。模型体积较小（通常在1B参数以下）。

代表模型：
1）Cohere Rerank (商业闭源标杆)：工业界公认最成熟的商业API，多语言支持与复杂语义扫描能力极强，是目前性能评估的上限参考。
2）BGE-Reranker（开源之光）：智源研究院出品（如bge-reranker-large）。且开源轻量，不仅业绩成绩近逼，而且非常适合企业本地损耗、低延迟部署。

适用场景：对系统延迟（Latency）要求较高的常规生产环境（运行约数十级）。

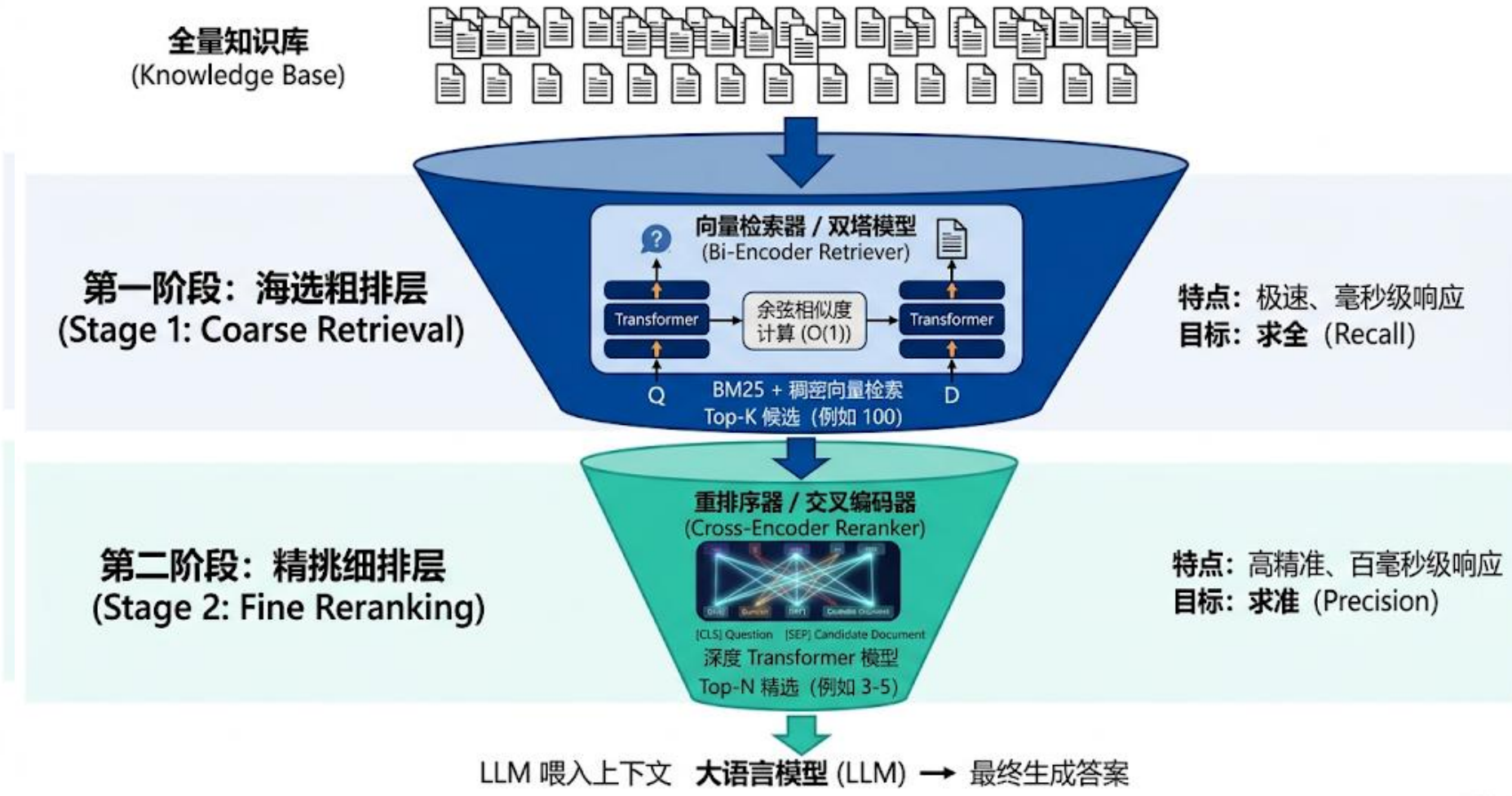
二：基于LLM的重排序

技术特点：颠覆了传统排序输出“分数”的思路，而是将排序任务转化为大模型最熟练的“语言生成任务”。通过设计专门的提示（如：“你是评分专家，请判断以下文档对回答问题的帮助程度”），利用千亿参数模型的零样本推理能力强制输出排序结果。

代表模型：
1）RankLLaMA / RankVicuna：专门基于大语言模型的开源排序模型。
2）直接调用GPT-4o / DeepSeek：使用通用最强大模型直接充当裁判。

适用场景：对延迟不敏感、对极估计逻辑推理要求极高的高价值场景（例如：紧急医疗诊断、法律合同比对），算力消耗极大。

RAG双级搜索与重排架构图



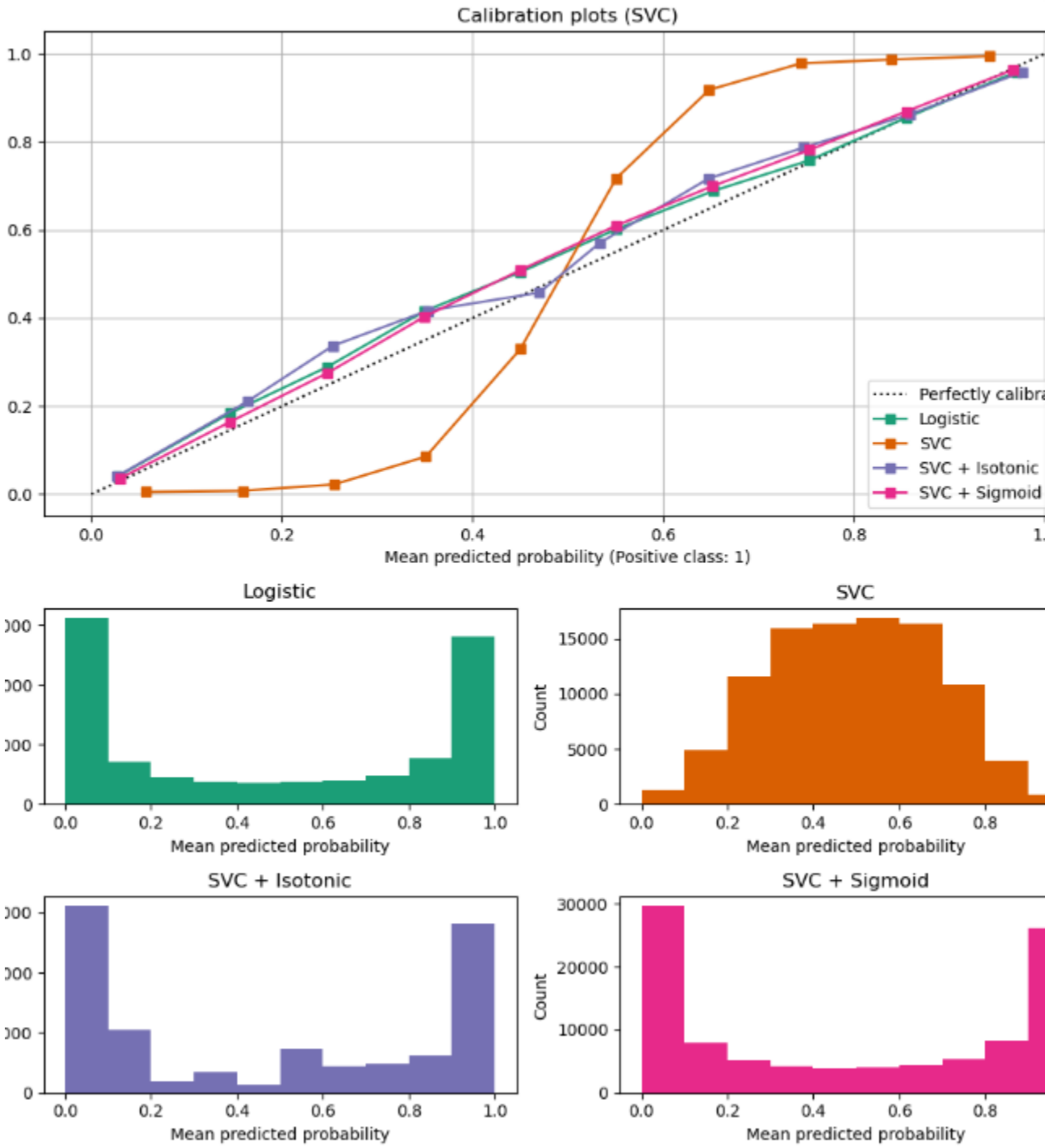
分数校准的理论意义

为何重排之后还要进行校准 (Calibration)?

无论是双塔模型的余弦相似度，还是交叉编码器输出的 Logits 值，它们本质上都是模型内部的标量，并非真实的概率。例如，得分 0.8 不等于“80%的可能性相关”。

如果当前知识库中压根没有正确答案，排在第一名的文档其实也是垃圾噪音。但因为它相对得分最高，依然会被 RAG 截取，进而引发大模型的严重幻觉。

分数校准的数学本质，是将原始得分映射为真实的条件概率分布 $P(\text{Relevant} \mid q, d) \in [0, 1]$。这赋予了系统设定“硬拒绝阈值”的能力，实现真正的“知之为知之，不知为不知”。



核心概率校准算法对比

算法名称	核心数学机制	技术优势与特点	在 RAG 架构中的应用建议
Platt Scaling (普拉特缩放 / Sigmoid映射)	在重排模型的原始输出后，串联一个**逻辑回归 (Logistic Regression)** 层，利用 Sigmoid 函数拟合。 $P = \frac{1}{1+\exp(A \cdot s + B)}$	**计算极快，泛化性好。** 能强行将无界的打分压缩进[0, 1]的标准概率区间。	**首选二分类校验：** 极其适合判断文档“相关”或“不相关”，是工业界最成熟的校准兜底方案。
Temperature Scaling (温度缩放)	在 Softmax 层输出前，将所有原始 Logits 分数统一除以一个**温度参数 T (T>1)**。 $P = \frac{1}{1+\exp(s/T)}$	**绝对保持排序不变。** 在完美软化“盲目自信”的同时，完全不会改变原有文档的排名先后顺序（维持 AUC 不变）。	**大模型/深层网络首选：** 实现成本极低，特别适合用于缓解千亿参数大语言模型作为裁判时的过度自信问题。
Isotonic Regression (保序回归)	拟合一个非参数的**阶梯式递增函数**，对模型输出的分数区间进行分段校正。	不假设分数的具体分布形状，能纠正模型任何形式的单调失真；但数据量少时易过拟合。	当系统积累了大量用户真实点击或反馈数据（>1000条）时，能达到比 Platt 缩放更精细的校准效果。

C-RAG 校正技术核心原理

C-RAG (Corrective RAG) 是一种“后检索+后生成”的双阶段优化技术，其核心是引入**检索结果校正**与**生成内容校验**的双闭环，彻底解决“检索偏差”与“大模型幻觉”问题。

核心第一步：检索校正 (源头控制)

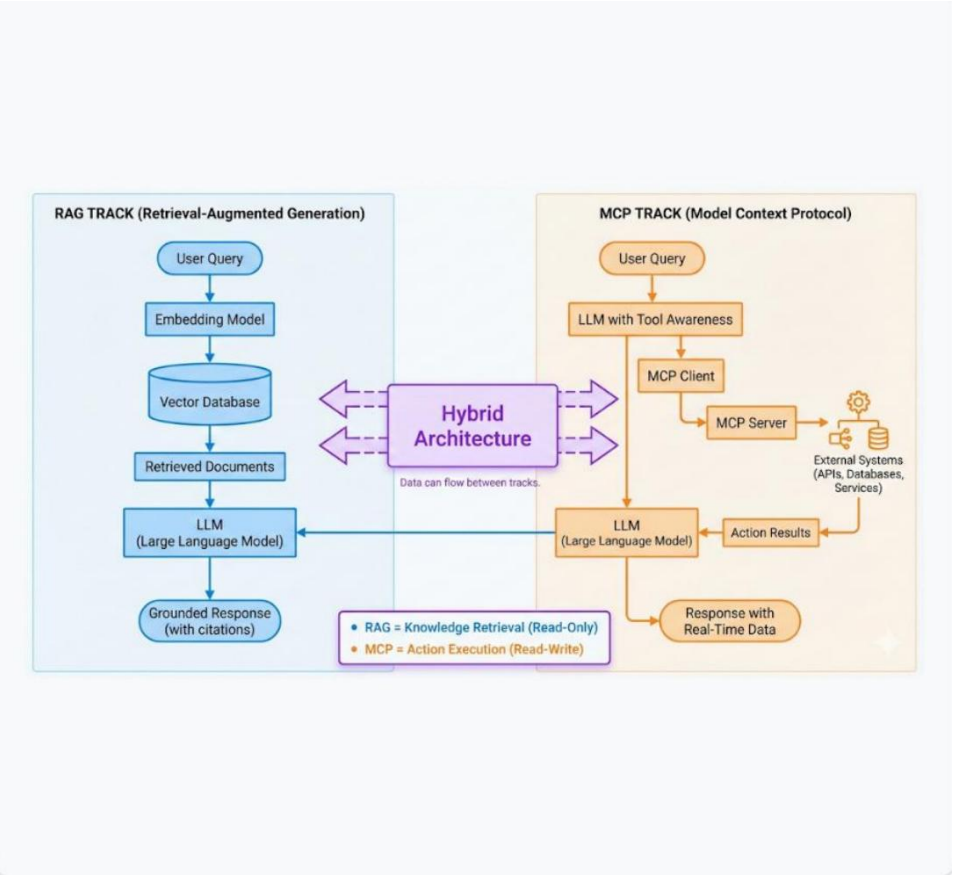
LLM 对原始检索结果进行严格的维度诊断：

- **校验维度：**分析文档的 相关性、完整性、准确性。
- **校正策略 (动态路由)：**
  - 无关结果：剔除并生成“补充检索指令”(Web Search)
  - 遗漏结果：生成“定向检索指令”补充细节
  - 错误结果：生成“修正检索指令”获取准确数据

核心第二步：生成校验 (末端拦截)

LLM 对初步生成的回答进行最后把关：

- **校验维度：**事实一致性、逻辑完整性、是否存在幻觉。
- **修正策略：**强制要求回答 100% 基于校正后的检索结果；自动删改大模型胡编乱造的虚假数据，补充缺失信息。



图：C-RAG“精准检索 → 可靠生成”流程示意图

C-RAG 与传统 RAG 核心差异对比

对比维度	传统 RAG (单向流水线)	C-RAG (双闭环校正范式)
错误处理	无主动校正，直接基于召回文档生成	**双环节主动校正**，从源头解决检索错误与生成幻觉
回答准确率	中等 (极易受检索噪音/错误信息影响)	**极高 (相比传统 RAG 提升 30%~50%)**
长尾/复杂场景	本地库无数据时，容易直接崩溃或产生幻觉	**鲁棒性强**，具备动态路由能力（内部校验+外部网络互补）
耗时与成本	响应快，Token消耗低，商用成本低	**相对较高** (新增两次 LLM 校验调用，响应时间增加 50%~100%)

◎ 核心结论与适用场景 (Trade-off)

C-RAG 的本质是用 **“算力成本/时间成本”** **换取** **“极致的准确率”**。

在实际工程落地中，它极其适配对**事实偏差零容忍**的专业级场景。例如：**企业级高管决策辅助库、金融法务数据检索、医疗政务问答。**面对这些场景，传统 RAG 的漏检和幻觉是致命的，而 C-RAG 的纠错机制能确保最终输出的绝对可靠。

资源与学习建议

一、 核心学术论文

- 《Lost in the Middle: How Language Models Use Long Contexts》 + https://arxiv.org/abs/2307.03172
- 《Passage Re-ranking with BERT》 + https://arxiv.org/abs/1901.04085
- 《On Calibration of Modern Neural Networks》 + https://arxiv.org/abs/1706.04599
- 《Corrective Retrieval Augmented Generation》 (C-RAG) + https://arxiv.org/abs/2401.15884
- 《From Local to Global: A GraphRAG Approach to Query-Focused Summarization》 + https://arxiv.org/pdf/2404.16130
- 《ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT》 + https://arxiv.org/pdf/2004.12832
- 《Precise Zero-Shot Dense Retrieval without Relevance Labels》(HyDE) + https://arxiv.org/pdf/2212.10496

二、 官方文档与工程实践资料

- Scikit-Learn 官方文档：1.16. Probability calibration + https://scikit-learn.org/stable/modules/calibration.html
- Sentence-Transformers 官方手册：Cross-Encoders vs Bi-Encoders + https://www.sbert.net/examples/applications/cross-encoder/README.html
- 工业界博客：Reranking in RAG: Cross-Encoders, Cohere Rerank & FlashRank + https://medium.com/@vaibhav-p-dixit/reranking-in-rag-cross-encoders-cohere-rerank-flashrank-c7d40c685f6a

感谢关注